

Advanced Programming Lab

[PCCCS495]

CAR RENTAL SYSTEM

Name: -Biswajit Kundu

Semester: -IV

Class Roll: - 49

Enrolment Number: - 12024052004047

Date: - 29/03/2026

**Instructor name: - Dr.Susovan Jana,
DR.Soumadip Biswas, Dr.Swagatam Basu**

Department of Information Technology
2026

1. Problem Statement

Managing car rentals manually is inefficient and prone to errors. Traditional systems lack proper tracking of available and rented vehicles, leading to confusion and incorrect bookings.

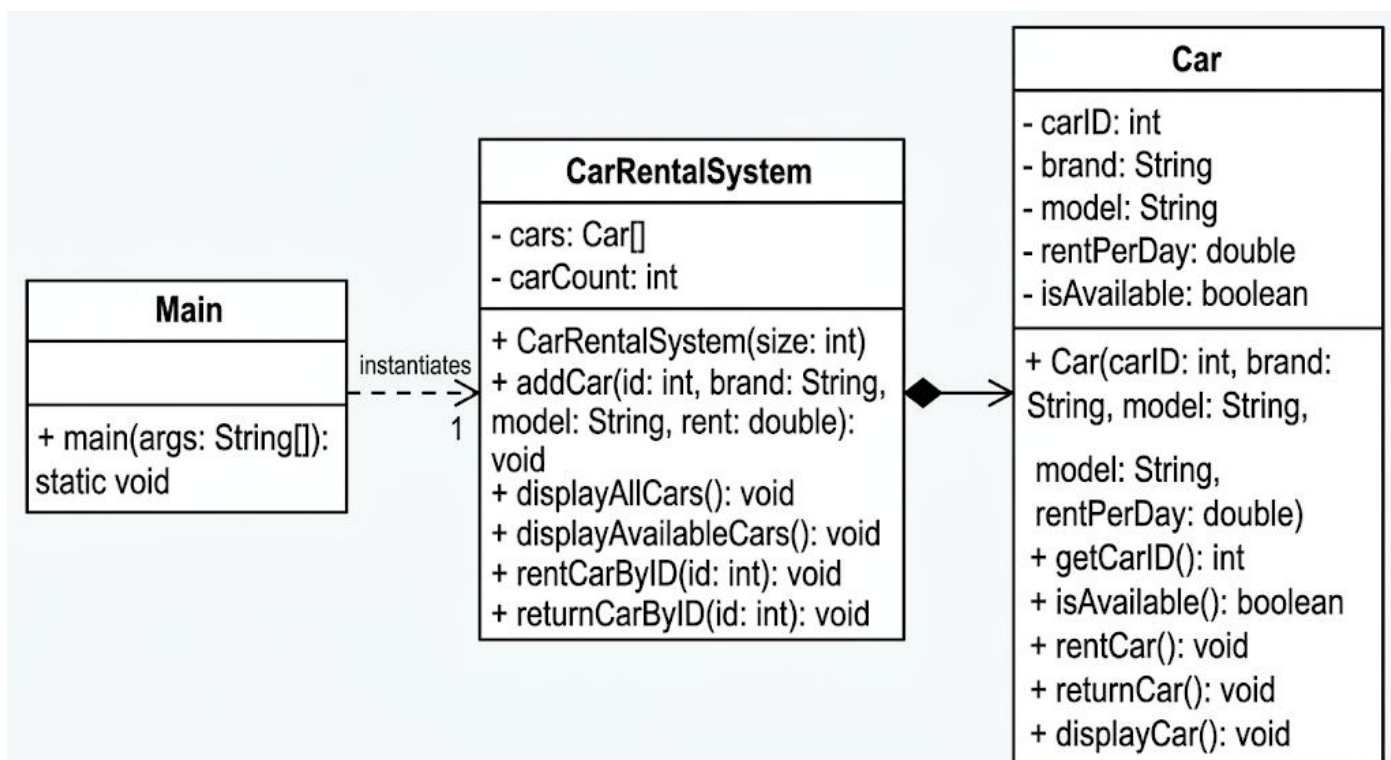
This project aims to develop a **console-based Car Rental System using Java**. The system allows users to:

- Add new cars
- View all cars
- Check available cars
- Rent cars using unique ID
- Return rented cars

The goal is to create a structured and efficient system using **Object-Oriented Programming (OOP)** principles.

2. System Design

2.1 UML Class Diagram:-



2.2 Description of Major Classes

i. Car Class

- Represents individual car objects
- Contains attributes like ID, brand, model, rent, and availability
- Provides methods to rent, return, and display car details

ii. CarRentalSystem Class

- Manages a collection of cars
- Handles operations such as adding, renting, returning, and displaying cars

iii. Main Class

- Entry point of the application
- Provides menu-driven interaction with the user

2.3 Interaction Flow

1. User starts the program
2. Menu is displayed
3. User selects an option:
 - Display all cars
 - Display available cars
 - Rent a car
 - Return a car
4. System processes request
5. Loop continues until exit

3. OOP Concepts Demonstrated

3.1 Abstraction

Where used: Car operations

Why used: Only essential functionalities (rent, return) are exposed

3.2 Encapsulation

Where used: Private data members in Car class

Why used: Protects data and ensures controlled access

3.3 Inheritance

Not implemented

Future scope: Can extend classes like ElectricCar, LuxuryCar

3.4 Polymorphism

Where used: Method behavior based on object state

Example: displayCar() shows different status (Available/Rented)

3.5 Exception Handling

Where used: Input validation using conditions

Prevents invalid operations like renting unavailable cars

4. Implementation Highlights

4.1 Adding Cars

```
system.addCar(101, "Toyota", "Innova", 2500);
```

Justification:

Adds cars dynamically into the system array.

4.2 Renting Logic

```
if (cars[i].isAvailable()) {  
    cars[i].rentCar();  
}
```

Justification:

Ensures only available cars can be rented.

4.3 Returning Logic

```
if (!cars[i].isAvailable()) {  
    cars[i].returnCar();  
}
```

Justification:

Prevents returning already available cars.

4.4 Availability Check

```
if (cars[i].isAvailable())
```

Justification:

Filters available cars for display.

5. Testing & Error Handling**5.1 Test Cases**

Test Case	Input	Expected Output
Rent Car	Valid ID	Car rented

Test Case	Input	Expected Output
Rent Car	Already rented	Error message
Return Car	Valid ID	Car returned
Invalid ID	Wrong ID	Not found

5.2 Edge Cases Handled

- No cars available
- Invalid car ID
- Renting already rented car
- Returning non-rented car

5.3 Failure Scenarios

- Wrong user input → handled via conditions
- Invalid menu choice → error message shown

6. Git Workflow

The project follows structured version control practices:

- Minimum 10 meaningful commits
- Each commit represents a logical step
- Clear commit messages
- Incremental development

7. Conclusion & Future Scope

The Car Rental System successfully demonstrates the implementation of Java and OOP concepts in a real-world scenario. It provides an efficient and structured way to manage rental operations using a simple console interface.

Future Enhancements

- Database integration (MySQL)
- GUI development (JavaFX/Swing)
- User authentication system
- Online booking system

Appendix

Approved Email: *Biswajit.Kundu2024@iem.edu.in*